COMP 395 — Deep Learning

Final Project: Applied Deep Learning

Groups of 1–3 • 4 Weeks • Due at End of Unit • 21% of Final Grade

1 Overview

This capstone project asks you to apply the deep learning knowledge you have built throughout the semester—from gradient descent and backpropagation through CNNs, RNNs/LSTMs, and Transformers—to a problem of your choosing. You may also choose to research and implement more ambitious frontier deep learning techniques that we have not covered in class. You will train and evaluate one or more deep learning models using PyTorch with GPU acceleration, write a short research-style report in LaTeX, and deliver a well-documented GitHub repository that a reviewer could clone and reproduce.

2 What You Will Do

Your group will select a problem, acquire or identify a dataset, design and implement a deep learning pipeline, run experiments with proper logging, and communicate your results. Concretely, every project must include the following:

- **Problem & dataset.** Choose a task (classification, generation, detection, translation, etc.) and either collect your own data or use an existing public dataset. If you collect data, document your collection and preprocessing pipeline.
- **Model training.** Train at least one deep learning model in PyTorch on a GPU. You may fine-tune a pretrained model (e.g., a HuggingFace Transformer) or train from scratch. Compare at least two meaningful experimental conditions (architecture variants, hyperparameters, data augmentation strategies, etc.).
- **Experiment tracking.** Log all runs with MLflow, Weights & Biases, or TensorBoard. Submitted training logs must include loss curves, evaluation metrics per epoch, and hyperparameter configurations.
- **Reproducible environment.** Your project must run from either a `Dockerfile` or a `uv`-managed environment with a lockfile. A reviewer should be able to set up and run your code with no more than three shell commands.
- **Report.** A ∼4-page LaTeX report (NeurIPS or IEEE two-column format) with sections: Introduction, Related Work, Methods, Results, Discussion, and Conclusion.
- **GitHub repository.** Clean, documented code with a comprehensive README (see Repository Requirements below).

3 Compute Resources

Most projects will require GPU acceleration. You have several options: your own machine (if it has a CUDA-capable GPU or Apple Silicon with MPS), Cloud resources (AWS, Azure, Vertex AI, etc.), or one of the instructor's lab machines. If you are thinking of using cloud resources, please let me know ahead of time and we can set you up with credits. If you plan to use a lab machine, coordinate scheduling with the instructor early. Regardless of where you train, your final submission must be reproducible from the Dockerfile or `uv` environment—Colab-only notebooks are not acceptable as the sole deliverable.

4 Repository Requirements

Your GitHub repository is a first-class deliverable. It should be clear enough that someone unfamiliar with your project could understand and run it. The README must include:

- **Project summary.** One-paragraph description of the problem, approach, and key results.
- **Codebase map.** A brief description of each major file or directory and its purpose.
- **Architecture diagram(s).** Visual representation of your model architecture(s). These can be created with draw.io, Mermaid, LaTeX TikZ, or any tool—hand-drawn and scanned is also fine.
- **Setup & run instructions.** Step-by-step commands to build the environment and reproduce training and evaluation.
- **Results summary.** A table or chart of your main results (can link to the report for details).
- **AI usage log.** A brief summary of how LLM/AI tools were used and what conversations are included in the repo (per course AI policy, transcripts must be submitted).

5 Report Format

Your report should be approximately 4 pages in a standard conference format (NeurIPS single-column or IEEE two-column). Use the following structure:

Section	Description
Introduction	Motivate the problem. What are you trying to do and why does it matter? State your research question or hypothesis.
Related Work	Summarize 3–5 relevant papers or projects. How does your work relate to or build on prior efforts? Cite properly.
Methods	Describe your model architecture(s), training procedure, dataset, preprocessing, and evaluation metrics. Include enough detail for reproducibility. Reference your architecture diagram.
Results	Present your quantitative results with tables and/or figures. Compare experimental conditions. Include training curves.
Discussion	Interpret your results. What worked? What didn't? Why? Discuss limitations and potential improvements.
Conclusion	Summarize your contributions and suggest future directions.

The report must be compiled from LaTeX source. Submit both the `.tex` source and the compiled `.pdf` in your repository.

6 AI & LLM Policy

Use of AI tools (ChatGPT, Claude, Copilot, etc.) is **encouraged** for this project—for debugging, boilerplate, environment setup, brainstorming, drafting, plotting code, and learning new APIs. However, the standard course policy applies in full:

Every group member must be able to explain any randomly selected part of the codebase and report. This will be assessed in a brief oral check during or after the final presentation. Inability to explain code you submitted is treated as a violation of academic integrity.

Include AI conversation transcripts in your repository (a `transcripts/` folder or links). In your README, briefly describe how AI was used and what value it provided.

7 Grading Rubric

The project is worth 21% of your final grade. The breakdown below applies per group, except for the oral explanation component, which is assessed individually.

Component	Weight	What We're Looking For
Technical Implementation	30%	Correct, well-structured PyTorch code. Meaningful experiments (not just one training run). Proper use of GPU. Code runs from the provided environment.
Report Quality	35%	Clear writing. Proper LaTeX formatting. All required sections present with substance. Figures and tables are informative and well-labeled. Thoughtful comparison of experimental conditions. Honest discussion of what worked and what didn't. Training curves and metrics properly presented.
Repository & Reproducibility	20%	Clean repo structure. Comprehensive README with architecture diagram and codebase map. Dockerfile or `uv` lockfile works. Training logs included.
Oral Explanation	10%	Each member can explain randomly selected parts of the code and report. Demonstrates genuine understanding, not just surface familiarity. **Graded individually.**

8 Example Project Ideas (Non-Exhaustive) applying concepts from class to real-world problems

You are free to propose any project that involves training a deep learning model. Here are some starting points to spark ideas—you are not limited to these:

- **Fine-tune a Vision Transformer (ViT)** on a domain-specific image classification task (medical imaging, satellite imagery, art style classification, etc.).
- **Build a text classifier** by fine-tuning a pretrained language model (BERT, DistilBERT, RoBERTa) on a dataset you curate—e.g., classifying Reddit posts, app reviews, or news headlines.
- **Train an image generation model** (diffusion model, VAE, or GAN) on a curated dataset and evaluate generation quality.
- **Multimodal project:** combine text and image features for a task like visual question answering, image captioning, or meme classification.
- **Collect your own dataset** (web scraping, manual labeling, sensor data) and train a model on it. Document your collection methodology thoroughly.
- **Reproduce and extend a paper:** pick a recent deep learning paper, reproduce its core result, then run an additional experiment the authors didn't try.
- **Sequence modeling:** music generation, time-series forecasting, code completion, or protein sequence classification using Transformers or LSTMs.

- **Object detection or segmentation on the edge** using a pretrained backbone (YOLO, DETR, Segment Anything) fine-tuned on a custom dataset. Then optimize for a NVIDIA Jetson Xavier edge device.

9 Frontier Project Ideas

The ideas below go beyond the standard CNN/RNN/Transformer toolkit and touch on architectures at the frontier of deep learning research. They are more ambitious, but well-scoped versions are absolutely achievable in four weeks—especially if you lean on existing open-source implementations as a starting point and focus your contribution on a clear experiment or extension. Each idea includes a suggested "starter scope" to help you right-size the project.

9.1 Neural Operators & PDE Solvers

Neural operators learn mappings between *function spaces* rather than finite-dimensional vectors, making them powerful tools for solving families of partial differential equations (PDEs) without re-running expensive numerical solvers for each new set of initial/boundary conditions.

- **Fourier Neural Operator (FNO) for fluid dynamics.** Train an FNO to predict the time evolution of 2D Navier-Stokes flow at varying viscosities using the benchmark dataset from Li et al. (2021). Compare the FNO against a standard CNN baseline (e.g., a U-Net) on both accuracy and inference speed. Investigate how performance degrades when you evaluate at spatial resolutions the model was not trained on (zero-shot super-resolution).

- **Physics-informed neural network (PINN) for a classical PDE.** Implement a PINN that embeds a known PDE (heat equation, Burgers' equation, or wave equation) directly into the loss function as a soft constraint. Train the network to satisfy both boundary/initial conditions and the PDE residual without any simulation data. Compare convergence and accuracy against a data-driven MLP baseline that is trained on numerical solver output.

- **DeepONet for operator learning.** Implement the Deep Operator Network (DeepONet) architecture—a branch network that encodes the input function and a trunk network that encodes the query location—and train it to learn the solution operator for a parametric ODE or PDE (e.g., antiderivatives, diffusion-reaction). Compare against an FNO on the same task.

9.2 Graph Neural Networks (GNNs)

Graph neural networks generalize deep learning to non-Euclidean, irregular structures—social networks, molecules, meshes, citation graphs, and more. The key primitive is *message passing*: each node aggregates information from its neighbors to update its representation.

- **Molecular property prediction.** Use a GNN (GCN, GAT, or GIN) to predict molecular properties (solubility, toxicity, or binding affinity) from molecular graphs where atoms are nodes and bonds are edges. Train on a benchmark dataset from MoleculeNet (e.g., ESOL, Tox21, BBBP). Compare at least two GNN variants and analyze which molecular substructures the model attends to.

- **Node classification on citation networks.** Implement and compare GCN, GraphSAGE, and GAT on a standard citation graph benchmark (Cora, CiteSeer, or PubMed). Go beyond

reporting accuracy: investigate how performance changes with graph depth (over-smoothing), vary the fraction of labeled nodes, and visualize learned node embeddings with t-SNE or UMAP.

- **Traffic or weather forecasting on spatial graphs.** Model a road network or weather station network as a graph and use a spatio-temporal GNN (e.g., STGCN, DCRNN) to forecast traffic speed or temperature at each node over time. Compare against a non-graph baseline (per-node LSTM or Transformer).

- **GNN-based physics simulation.** Train a GNN to simulate particle-based or mesh-based physics (e.g., predicting the next state of a system of interacting particles). Use the approach from Sanchez-Gonzalez et al. (2020) where particles are nodes and interactions are edges. Evaluate rollout stability over multiple timesteps.

9.3 Spiking Neural Networks (SNNs)

Spiking neural networks are inspired by biological neurons: instead of continuous activations, neurons communicate through discrete *spikes* over time. They are of growing interest for energy-efficient neuromorphic computing and for modeling temporal data with fine-grained timing.

- **SNN image classification with surrogate gradients.** Implement a spiking CNN using leaky integrate-and-fire (LIF) neurons and train it on MNIST or Fashion-MNIST using surrogate gradient descent (since the true spike function is non-differentiable). Compare accuracy and the number of spikes (a proxy for energy cost) against a conventional CNN of similar capacity.

- **Temporal pattern recognition with SNNs.** Use an SNN to classify temporal spike-train data—for example, the Spiking Heidelberg Digits (SHD) or DVS gesture datasets, where input is already event-based. Compare a recurrent SNN against an LSTM baseline on the same task. Analyze how the SNN's temporal coding captures timing information that rate-coded networks might miss.

- **ANN-to-SNN conversion.** Train a conventional CNN to high accuracy on CIFAR-10, then convert it to a spiking network by replacing ReLU activations with integrate-and-fire neurons. Measure the accuracy–latency tradeoff: how many simulation timesteps does the SNN need to match the ANN's accuracy? Experiment with threshold balancing and normalization techniques to minimize the conversion gap.

9.4 Kolmogorov-Arnold Networks (KANs)

Kolmogorov-Arnold Networks replace the fixed activation functions of standard MLPs with *learnable univariate functions* on each edge of the network, inspired by the Kolmogorov-Arnold representation theorem. Instead of learning weight matrices applied to fixed nonlinearities, KANs learn the nonlinearities themselves (typically parameterized as B-splines). This can yield more interpretable and parameter-efficient models, especially on scientific and mathematical tasks.

- **KAN vs. MLP on symbolic regression.** Compare a KAN against a standard MLP on learning known mathematical functions (e.g., $f(x, y) = \sin(x) + y^2$, Bessel functions, or physical laws like $F = ma$). Evaluate which architecture recovers the true functional form more faithfully and with fewer parameters. Visualize the learned activation functions on each edge to assess interpretability.

- **KAN for PDE surrogate modeling.** Use a KAN as the backbone of a physics-informed neural network (PINN) and compare it against the standard MLP-based PINN on a classical PDE (Burgers', Poisson, or advection). The hypothesis from Liu et al. (2024) is that KANs' learnable activations can capture sharp gradients and discontinuities more efficiently. Test this claim.
- **KAN on tabular / scientific data.** Train a KAN on a real-world scientific or tabular dataset (e.g., predicting material properties, astronomical measurements, or a Kaggle tabular benchmark) and compare against an MLP, XGBoost, and/or a small Transformer. Investigate whether the learned edge activations reveal meaningful nonlinear relationships in the features.

These ideas are more specialized than the core project suggestions. If you pursue one, budget extra time in Week 1 for reading the relevant paper(s) and getting the library ecosystem set up. The "starter scope" suggestions are designed to be achievable in four weeks—start there and extend if time allows.

10 Suggested Timeline

Week	Milestone	Details
1	Proposal & Setup	Form group. Submit a 1-paragraph proposal (problem, dataset, planned approach). Set up repo, Dockerfile/`uv` environment, and verify GPU access.
2	Data & Baseline	Finalize dataset and preprocessing pipeline. Train a baseline model. Begin experiment logging.
3	Experiments	Run your main experiments (architecture comparisons, hyperparameter sweeps, ablations). Begin drafting the report.
4	Write-Up & Submission	Finalize report. Clean up repo and README. Ensure reproducibility. Submit repo, report PDF, and training logs. Oral checks conducted this week.

11 Deliverables Checklist

☐ GitHub repository (link submitted via GitHub Classroom)

☐ LaTeX report: `.tex` source + compiled `.pdf` (∼4 pages, in repo)

☐ Training logs (MLflow / W&B / TensorBoard exports, in repo or linked)

☐ `Dockerfile` or `uv.lock` / `pyproject.toml` at repo root

☐ README with project summary, codebase map, architecture diagram, setup instructions, results summary, and AI usage log

☐ AI conversation transcripts (in `transcripts/` folder or linked)

☐ Week 1 proposal (1 paragraph, submitted separately or as initial commit)

Questions? Reach out to the instructor during office hours or on the course communication channel. Start early, scope wisely, and have fun with this—this is your chance to build something you're proud of.